API SECURITY ANALYSIS REPORT

Project Title	API Security Analysis
Organization	Future Interns
Prepared by	Sanita Salomon
Date	15 Mar 2026

Tools Used

Postman	Browser Dev Tools

Introduction

Application Programming Interfaces (APIs) play a critical role in modern web applications by enabling communication between different software systems and services. APIs allow applications to exchange data and perform operations through defined endpoints, typically using HTTP requests and responses formatted in JSON. While APIs provide flexibility and efficiency, they can also introduce security risks if they are not properly designed and protected.

The purpose of this report is to analyze the security aspects of a public test API provided by JSONPlaceholder. The analysis focuses on several API endpoints, including /posts, /users, and /posts/{id}, in order to understand how data is exposed and how the API handles authentication and access control. Browser developer tools were used to inspect network requests, response headers, and API responses.

This report identifies common API security risks such as lack of authentication, excessive data exposure, predictable resource identifiers, and potential vulnerabilities such as Insecure Direct Object References (IDOR). In addition, the analysis includes an evaluation of HTTP headers and API documentation to understand the structure and behavior of the API. Based on these observations, appropriate remediation strategies and best practices are suggested to improve the overall security of API implementations.

(A) API Security Analysis – /posts Endpoint

- API URL: https://jsonplaceholder.typicode.com/posts

1. Observed Behavior

The /posts endpoint returns a list of posts in JSON format. Each object contains the following fields:

- userId – ID of the user who created the post
- id – Unique identifier for the post
- title – Title of the post
- body – Content of the post

The API returns multiple records (100 posts) in a single response.

2. Authentication and Access Control Assessment

No authentication mechanism is required to access this endpoint. Anyone can send a request to the API and retrieve the full list of posts.

There are no visible security controls such as:

- API key
- Token authentication
- OAuth
- Session validation

This means the API is publicly accessible without identity verification.

3. Common API Security Risks Identified

1. Lack of Authentication

The API allows anyone to access the data without authentication.

Risk:
Unauthorized users could retrieve large amounts of data.

2. Excessive Data Exposure

The endpoint returns all posts at once (100 records).

Risk:
In a real application, this could expose unnecessary information and increase the risk of data leakage.

3. No Pagination or Rate Limiting

The API response does not show pagination or request limits.

Risk:
Attackers could repeatedly send requests and potentially overload the server.

4. Predictable Object Identifiers

Each post has a predictable numeric ID (1–100).

Risk:
Attackers could easily enumerate objects by changing the ID value in requests.

4. Business Impact (Simple Explanation)

If similar issues existed in a real business API:

- Unauthorized users could access company data.
- Attackers could collect large amounts of information automatically.
- The system might become slow or unavailable due to excessive requests.
- Sensitive data could be exposed to competitors or malicious actors.

5. Recommended Remediation Steps

(a) Implement Authentication

Require authentication using:

- API keys
- JWT tokens
- OAuth authentication

(b) Apply Access Control

Ensure users can only access data that belongs to them.

(c) Implement Pagination

Limit responses using pagination such as:

/posts?page=1&limit=10

This prevents excessive data exposure.

(d) Add Rate Limiting

Restrict the number of requests per user to prevent abuse and denial-of-service attacks.

(e) Validate and Monitor API Requests

Log API activity and monitor unusual traffic patterns to detect attacks.

You can place the following **analysis under the /users endpoint screenshot** in your report.

(B) API Security Analysis – /users Endpoint

- API URL: https://jsonplaceholder.typicode.com/users

1. Observed Behavior

The /users endpoint returns a list of user records in JSON format. Each user object contains several fields including:

- id – Unique identifier for the user
- name – Full name of the user
- username – User login name
- email – User email address
- phone – Contact phone number
- website – User website
- address – Includes street, city, zipcode, and geographical coordinates
- company – Contains company name and business details

The API returns 10 user records in a single response with nested data structures such as address and company information.

2. Authentication and Access Control Assessment

The /users endpoint can be accessed without any authentication mechanism.

There is **no evidence of security controls such as:**

- API key authentication

- Token-based authentication (JWT)
- OAuth authorization
- Role-based access control

This means that any user can access personal user information directly from the API.

3. Common API Security Risks Identified

1. Exposure of Personal Information

The API exposes sensitive user-related data such as:

- Email addresses
- Phone numbers
- Physical addresses
- Geographical coordinates

Risk:
In a real application, exposing such data without protection could lead to privacy violations or targeted attacks.

2. Lack of Authentication

The endpoint allows unrestricted access to user data.

Risk:
Unauthorized users could collect personal information from the system.

3. Excessive Data Exposure

The API returns detailed nested information including address and company data.

Risk:
Returning more data than necessary increases the chance of sensitive data leakage.

4. Predictable User IDs

User records use sequential numeric identifiers (1–10).

Risk:
Attackers could attempt enumeration attacks by manipulating user IDs to retrieve additional records.

4. Business Impact

If this issue existed in a real business system:

- Customer personal information could be exposed publicly.
- Attackers could collect email addresses and phone numbers for phishing attacks.
- Privacy laws and compliance requirements could be violated.
- The organization's reputation and customer trust could be damaged.

5. Recommended Remediation Steps

(a) Implement Strong Authentication

Require authentication before accessing user information using:

- API keys
- JWT tokens
- OAuth authentication

(b) Apply Access Control Policies

Ensure that users can only access their own data or data they are authorized to view.

(c) Limit Data Exposure

Return only necessary fields instead of full user profiles when possible.

Example:

```
{
  "id": 1,
```

"name": "Leanne Graham"

}

(d) Add Rate Limiting

Limit the number of API requests per user to prevent automated data scraping.

(e) Protect Sensitive Information

Sensitive data such as addresses, phone numbers, and emails should only be accessible to authorized users.

(C) API Security Analysis – /posts/{id} Endpoint

- API URL: https://jsonplaceholder.typicode.com/posts/1

1. Observed Behavior

The /posts/{id} endpoint returns a single post object based on the provided post ID. In this case, the API request /posts/1 returns the details of post ID 1.

The JSON response contains the following fields:

- userId – ID of the user who created the post
- id – Unique identifier of the post
- title – Title of the post
- body – Content of the post

The API retrieves the specific resource using the ID value included in the request URL.

2. Authentication and Access Control Assessment

The endpoint can be accessed without authentication. Any user can retrieve post data simply by specifying the post ID in the URL.

There is no evidence of authentication or authorization controls, such as:

- API key validation

- Token-based authentication (JWT)
- OAuth authentication
- Role-based access control

This means the endpoint allows public access to individual records.

3. Common API Security Risks Identified

1. Insecure Direct Object Reference (IDOR)

The API allows direct access to resources by changing the ID value in the URL.

Example:

/posts/1

/posts/2

/posts/3

Risk:
Attackers can enumerate resources by modifying the ID value and retrieve large amounts of data.

2. Lack of Authentication

The endpoint does not require authentication before accessing data.

Risk:
Unauthorized users could retrieve information from the system without verification.

3. Predictable Resource Identifiers

The post IDs are sequential numbers.

Risk:
Attackers can easily guess valid IDs and perform automated data extraction.

4. No Request Rate Limiting

The API does not show any visible protection against excessive requests.

Risk:
Attackers could repeatedly request resources, potentially causing server performance issues.

4. Business Impact (Simple Explanation)

If similar vulnerabilities existed in a real business system:

- Attackers could retrieve sensitive records by manipulating IDs.
- Large amounts of internal data could be collected automatically.
- Customer information or private content could be exposed.
- The organization could face data security and compliance risks.

5. Recommended Remediation Steps

(a) Implement Authentication

Require users to authenticate before accessing API resources using secure methods such as:

- API keys
- JWT tokens
- OAuth authentication

(b)Apply Authorization Checks

Verify that the user has permission to access the requested resource.

(c) Use Non-Predictable Identifiers

Replace sequential numeric IDs with more secure identifiers such as UUIDs.

Example:

/posts/9f3a8c7e-41b2-4c9a-a4e2-8f34b5c7a921

(d)Implement Rate Limiting

Limit the number of requests from a user or IP address to prevent automated attacks.

(e) Monitor API Usage

Log and monitor API requests to detect suspicious behavior or repeated access attempts.

Based on the screenshots provided from your browser DevTools, here is a detailed analysis and report of the network request.

(D) Website Header Analysis Report

1. General Request Information

- **Request URL:** https://jsonplaceholder.typicode.com/
- **Request Method:** GET
- **Status Code:** 304 Not Modified
    - *Note: This indicates the browser used a cached version of the resource because the content hasn't changed since the last request.*
- **Remote Address:** 104.21.59.19:443 (Cloudflare CDN)
- **Referrer Policy:** strict-origin-when-cross-origin

2. Security & Header Observations

The analysis reveals several key headers related to security, infrastructure, and performance:

- **Infrastructure Disclosure:**
    - Server: cloudflare
    - Via: 2.0 heroku-router
    - X-Powered-By: Express
    - *Observation:* These headers explicitly reveal the backend stack (Node.js/Express) and the hosting environment (Heroku). In a hardened production environment, these are often hidden to prevent "fingerprinting" by attackers.

- **CORS (Cross-Origin Resource Sharing):**
    - Access-Control-Allow-Credentials: true
    - *Observation:* This allows the browser to share cookies or TLS client certificates across origins if the origin is explicitly trusted.
- **Rate Limiting:**
    - X-Ratelimit-Limit: 1000
    - X-Ratelimit-Remaining: 999
    - *Observation:* The API has active rate-limiting in place to prevent abuse or DoS attacks, allowing 1,000 requests per window.

3. Caching & Performance

- **Cache Status:** HIT (via Cloudflare)
- **Cache-Control:** public, max-age=43200 (The resource is cacheable for 12 hours).
- **Etag:** W/"2147-19cee71d187" (A weak validator used to check if the cached version is still current).
- **Alt-Svc:** h3=":443"; ma=86400 (Indicates support for HTTP/3).

4. Error Tracking & Reporting

The screenshots show active Network Error Logging (NEL) and reporting endpoints:

- **Report-To:** Points to https://nel.heroku.com/reports
- **Reporting-Endpoints:** Confirms that the server is configured to send reports regarding connection failures or security violations to a Heroku-managed endpoint.

Summary of Vulnerabilities/Risks

Risk Level	Observation	Recommendation
Low	**Information Exposure:** The X-Powered-By and Via headers reveal the technology stack.	Disable the X-Powered-By header in the Express configuration.
Low	**Missing Security Headers:** Common headers like X-Content-Type-Options, Content-Security-Policy (CSP), or Strict-Transport-Security (HSTS) were not visible in these specific frames.	Audit the server config to ensure modern security headers are being sent.

(E) API Documentation

Description

The API documentation for JSONPlaceholder was reviewed to understand the available endpoints, request methods, and data structures. The documentation provides information about different resources such as posts, users, comments, albums, photos, and todos.

By reviewing the documentation, it was possible to identify the correct API endpoints and understand how requests should be structured. This helped in selecting specific endpoints for testing and analyzing the API responses during the security assessment.

The documentation also explains the expected JSON format returned by the API, which assisted in interpreting the response data and identifying potential security considerations such as exposed fields and publicly accessible resources.

Conclusion

This report presented a security analysis of several endpoints provided by the JSONPlaceholder API. The analysis demonstrated how API responses can be examined using browser developer tools to understand the structure of returned data, the security controls in place, and potential vulnerabilities that may exist in API implementations.

The findings indicate that the analyzed endpoints allow unrestricted access to data without requiring authentication or authorization. Additionally, the APIs expose multiple records in a single response and use predictable identifiers, which could potentially allow attackers to enumerate resources or collect large amounts of data. Although JSONPlaceholder is designed as a public testing API and does not contain real sensitive information, these observations highlight security risks that could be significant in real-world applications.

The analysis also examined HTTP headers and infrastructure details revealed through the network requests. While some protections such as rate limiting and caching were observed, certain information disclosure and missing security headers were noted. Implementing stronger authentication mechanisms, proper access control, rate limiting, and minimizing data exposure are important steps in securing APIs.

Overall, this assessment demonstrates the importance of secure API design and proper configuration to protect systems and user data. Organizations should follow established API security best practices to reduce the risk of data exposure, unauthorized access, and other security threats.